Take-Home Assignment: Motor Characterization Bench

Automation & Test Software Engineer — Humanoid Robotics

Time budget: 10–16 hours of focused work. We value clean architecture and robust error handling over feature completeness. A working demo covering one test phase with solid parsing is better than a feature-complete submission with fragile code.

Overview

Build a cross-platform GUI application in **Python** that replays and processes data from a motor characterization test bench. The bench uses three instruments—a BLDC motor controller, a power supply, and a torque sensor—each producing data at a different rate and in a different format. Your application reads pre-recorded data files from all three, synchronizes them by timestamp, executes a test state machine, and produces a unified output log.

The instrument protocols and test parameters are defined in two YAML configuration files provided with this assignment. **Your parser and state machine must read their behavior from these YAML files**—do not hardcode packet layouts, field offsets, thresholds, or phase parameters.

Evaluation notice: We will test your submission against **different data files** than the ones provided.

Provided Files

Configuration Files

File	Purpose
`motor_protocol.yaml`	Fully defines the binary packet structure: framing, header layout, type definitions (with Python `struct` format strings), checksum algorithm, response field definitions, and command vocabulary.
`test_config.yaml`	Defines the test state machine: phase names and parameters, safety thresholds, PSU command templates, data source format descriptions, sync method, and output columns.

These YAML files are the specification. This PDF provides context and explains what we expect—the YAMLs define the exact technical details your code must implement.

Data Files

File	Format	Rate	Duration	Description
`test_motor_1000hz.bin`	Binary	1,000 Hz	5.0 s	5,000 telemetry packets. Structure defined in `motor_protocol.yaml`.
`test_motor_1000hz.csv`	CSV	1,000 Hz	5.0 s	Same data as the `.bin`, pre-decoded. 3 columns.
`test_sensor_4800hz.csv`	CSV	4,800 Hz	∼5.1 s	Torque readings with real-world timestamp jitter.
`test_psu_10hz.csv`	CSV	∼10 Hz	∼5.2 s	PSU voltage and current measurements with polling jitter.

All files cover approximately the same 5-second window, The binary and CSV motor files contain identical data—use the csv to validate your binary parser.

Architecture Requirements

YAML-Driven Parsing (Critical)

Your binary parser must construct its unpacking logic by reading `motor_protocol.yaml` at startup. Specifically:

- Read the `framing` section to identify start/end markers, header field offsets and types, checksum algorithm, and payload location.
- Read the `types` section to map type names to `struct` format strings and byte sizes.
- Read the `responses` section to build the field layout for each response code (field names, types, units).
- Read `test_config.yaml` to configure the state machine phases, thresholds, output columns, and sync method.

Design test: If we changed the telemetry payload tomorrow—adding a `temperature` field to the YAML—your parser should handle it without code changes.

Driver Abstraction

The motor controller provides data in two formats (binary and CSV). Your application must support both through a common interface:

```
MotorDataSource (interface)
  |-- MotorBinaryReader   -- parses .bin using motor_protocol.yaml
  |-- MotorCSVReader      -- parses .csv using test_config.yaml column defs

SensorDataSource (interface)
  |-- SensorCSVReader

PSUDataSource (interface)
  |-- PSUCSVReader
```

The GUI provides a file picker for each source. For the motor, a format selector (Binary / CSV) determines which reader is instantiated. The state machine and synchronization logic interact only with the interfaces.

Data Synchronization

Three streams at three rates (1000 Hz, 4800 Hz, ~10 Hz) must be combined. The motor stream is the **primary clock**. For each motor sample, find the most recent sensor and PSU readings with timestamps $\leq$ the motor timestamp. This is a *nearest-prior join*. For extra credit - extrapolate other data to fit the timestamp.

On jitter: The sensor timestamps are not perfectly periodic—they have real-world scheduling jitter (typically $\pm 20\,\mu$s, occasionally up to ± 0.5 ms). A sync algorithm that assumes uniform spacing will produce incorrect alignments.

Measured vs. Commanded Values

The output log contains two classes of PSU data. **Measured** values (`psu_voltage_v`, `psu_current_a`) come from the PSU CSV file—these represent what the instrument actually reported. **Commanded** values (`commanded_voltage_v`, `commanded_current_a`) are computed by the state machine based on the test phase and YAML parameters—these represent what the automation *would have sent* to the hardware.

Test Procedure

The test procedure is defined in `test_config.yaml` under the `test.phases` section. Your state machine must read its phase names, parameters, and transition logic from this configuration. Conceptual flow:

```
SETUP --> CURRENT_RAMP --> TORQUE_HOLD --> VOLTAGE_DECREASE --> COMPLETE
```

SETUP: Load and validate all data files. Report file stats (row counts, time spans, any immediate parse errors). Initialize state variables from YAML.

CURRENT_RAMP: Read motor and sensor samples. Track commanded current ramping at 3.4 A/s (derived from `max_current / ramp_duration` in YAML). Transition to TORQUE_HOLD when $|\text{torque}| \geq$ `target_torque` or $|\text{current}| \geq$ `max_current`.

TORQUE_HOLD: Hold commanded current at the value reached during ramp. Continue reading and logging all streams. Transition after `hold_duration_s` of data timestamps.

VOLTAGE_DECREASE: Track commanded voltage decreasing at the rate from YAML. Continue reading all streams. Transition when voltage $\leq$ `min_voltage` or data ends.

COMPLETE: Flush all buffers. Write the output CSV. Display summary statistics.

Timestamp-driven: Phase transitions are based on data timestamps, not wall clock. The same logic works at any playback speed or in batch mode.

End-of-data: The provided files cover ∼5 seconds, which is shorter than a full test run. Your state machine must handle data exhaustion gracefully in any phase—transition to COMPLETE, flush all buffers, and report which phase was active when data ended.

Safety & Error Handling (Required)

Safety Checks

Even in file replay, safety logic must be implemented—it validates that the state machine would protect hardware in a live scenario. Thresholds are in `test_config.yaml` → `test.safety`.

- **Torque limit:** If |torque| exceeds `max_torque_nm`, immediately transition to COMPLETE and log the abort reason.
- **Current limit:** Commanded current must never exceed `max_current_a`.
- **Manual abort:** A GUI button that forces transition to COMPLETE.

Data Error Handling

Your application must handle bad data gracefully—log the error, skip it, continue, and report a summary at the end.

Error Type	Expected Behavior
Binary: checksum failure	Discard packet, increment error counter, continue parsing.
Binary: missing or shifted start marker	Scan forward byte-by-byte to next valid `0xAA 0x55` sequence, realign parser.
Binary: truncated packet at end of file	Discard the partial packet, report incomplete read in summary.
Binary: unknown response code	Skip the payload (use `payload_size` to advance past it), log a warning.
CSV: malformed row (wrong field count, non-numeric values)	Skip the row, log a warning with row number, continue.
CSV: timestamp gap or non-monotonic timestamps	Continue with available data. Do not fabricate or interpolate missing samples.
CSV: empty file or missing headers	Report error at startup. Prevent test from running.
YAML: missing required field	Report error at startup with the full field path (e.g., `framing.start_marker.bytes`).

GUI Requirements

Use any Python GUI framework (Qt/PySide, Tkinter, web-based, etc). We do not evaluate visual polish.

Connection Panel

For each data source: a file picker and a format selector (Binary / CSV for motor; CSV for sensor and PSU). Display status after loading: row/packet count, time span, parse errors detected during initial scan. The test start button is enabled only when all three sources are loaded.

Test Control

Start button, Abort button, and playback speed control (1×, 5×, 10×, Max). At 1× the state machine advances in real time. At Max it processes as fast as possible.

Live Display

During playback: a time-series plot showing motor current (measured + commanded), torque, and PSU voltage. Current test phase should be visible. Update rate $\leq 20\,\text{Hz}$ regardless of playback speed.

Summary Panel

After completion: samples processed per source, parse errors per source, phase durations, peak torque, peak current, output file path.

Technical Constraints

Concurrency

File reading, parsing, synchronization, and GUI updates must not block each other. Document your threading/async model in `ARCHITECTURE.md`.

Performance

The provided files are small (∼100 KB binary, ∼24K rows CSV). In production these can be hundreds of MB. Use pre-compiled `struct.Struct` objects and avoid per-packet memory allocation. Track and report parsing throughput.

Efficiency Calculation (Bonus)

If time permits, compute and plot real-time mechanical efficiency. The formula is in `test_config.yaml` → `output.efficiency`:

$$\eta = \frac{\tau \cdot \omega}{V \cdot I}$$

Handle division by zero when PSU values are zero during setup or after completion.

Submission

Repository Structure

Your repository must clearly separate three concerns: **data source drivers** (parsing and reading), **test automation** (state machine, synchronization, safety), and **UI**. We suggest the following layout:

```
your-repo/
|-- config/           # Provided YAML files (do not modify)
|   |-- motor_protocol.yaml
|   |-- test_config.yaml
|-- drivers/          # Data source drivers (abstract + concrete)
|   |-- motor.py      # MotorDataSource, MotorBinaryReader, MotorCSVReader
|   |-- psu.py        # PSUDataSource, PSUCSVReader
|   |-- sensor.py     # SensorDataSource, SensorCSVReader
|-- automation/       # State machine, safety, synchronization
|-- ui/               # GUI code
|-- tests/            # Unit tests
|-- data/             # Provided data files
|-- AI_LOG.md
|-- ARCHITECTURE.md
|-- README.md
```

You may organize differently if your **ARCHITECTURE.md** justifies the choice. However, we must be able to import your driver classes from a `drivers` module—our evaluation harness depends on this.

AI Usage Log

Document all AI tool usage. You **must** highlight where you manually corrected AI-generated logic, especially regarding threading, timing, and data alignment. Be prepared to walk through your code and modify it live during the technical interview.

Unit Tests

At minimum, provide tests for:

- Binary parser: correct decode, checksum failure, partial packet, resync after corruption
- CSV parsers: normal read, malformed rows, empty file, missing columns
- YAML-driven construction: parser correctly builds field layout from YAML
- Synchronization: timestamp alignment with jittered data
- State machine: phase transitions, abort conditions

Evaluation Criteria

Priority	Criterion	What we look for
Must-pass	YAML-driven parsing	Binary parser reads `motor_protocol.yaml` at startup. State machine reads `test_config.yaml` for parameters. Adding a YAML field does not require code changes.
Must-pass	Parsing robustness	Handles corruption, truncation, resync, malformed CSV. Never crashes on bad input. Reports error statistics.
Must-pass	Driver abstraction	Motor binary and CSV readers share an interface. State machine has zero format-specific code.
Must-pass	Safety logic	Torque/current limits checked. Abort is deterministic.
Strong signal	Data synchronization	Correct nearest-prior join across three rates. Works correctly with jittered timestamps.
Strong signal	Architecture	Clean separation: drivers / logic / UI. Threading model documented and correct.
Bonus	Live visualization	Plots without degrading timing. Speed control works.
Bonus	Efficiency calculation	Correct formula from YAML. Division by zero handled.

A partially complete submission with clean YAML-driven architecture and robust error handling is valued more highly than a feature-complete submission with hardcoded offsets or fragile parsing. If short on time, submit what you have with a note on next steps.

Context

This assignment reflects a simplified version of our actual test infrastructure. The production system uses live serial connections (8 Mbps binary UART, SCPI instruments), USB-HID sensors via subprocess, and YAML-defined protocols for all motor communication. The core skills tested here—YAML-driven protocol parsing, multi-rate data synchronization, deterministic state machines, and defensive programming—are used daily.

SCPI Reference

The `test_config.yaml` includes SCPI command templates under `power_supply.commands`. In this file-replay assignment you do not send these commands—PSU data comes from the CSV file. The templates are included so your state machine can log what *would* be sent at each phase transition, and for reference if you later extend the application to control live hardware.